

Implementation Brief III: Operational Integrity

A complete synthesis of the five dependency-ordered phases, plus the optional seam, that gave the engine an event history, a memory of its own runs, a provenance graph, a sense of time, and a retrieval planner — without touching what already worked.

33	5 + 1	7	6	503	0
COMMITTS	PHASES & SEAM	NEW MODULES	NEW TABLES	TESTS PASSING	REGRESSIONS

CONTENTS

- A

Event fabric
- B

Manifests & replay
- C

Provenance graph
- D

Bitemporal canon
- E

Retrieval planner
- +

Principal seam
- §

Principles

Each phase depended on the last: **B**'s registries feed **C**'s supply-chain scoring; **C**'s edges feed **D**'s "why did this change" and **E**'s contradiction retrieval; **D**'s as-of queries feed **E**'s temporal retrieval. Every phase began with a dossier — real files and symbols cited, no code written — and stopped for explicit authorization before a single line changed.

A

Unified operational event fabric

Only the server's `/cycle` route ever produced a structured event — the CLI, where almost everything actually happens, produced none. Phase A gave every operation, wherever it's triggered from, one identical event stream.

5 commits · **0d88b3b** → **ed11bf5**

WHAT SHIPPED

- `domain_events` — an append-only table with no update or delete path, enforced by `EventRepository` simply exposing no such method. Every row carries a `correlation_id` (one operation's whole chain) and a `causation_id` (the specific event that triggered it).
- `OperationEmitter` — tracks one operation's correlation and causation pointer so `cycle.py` / `multicycle.py` / `council.py` / `dream.py` never thread a "last event id" variable by hand.
- A coarse actor taxonomy — `cli`, `server`, `dream`, `system` — deliberately not an identity model (that's the seam, below).
- `canon_events` stayed exactly as it was: `record_canon_event` became the one place that cross-emits into `domain_events`, so the two histories never compete.

The gap wasn't "no logging" — it was two histories that could disagree. Centralizing emission in the domain functions themselves, not the CLI or the engine facade, was what let `RetrievalCompleted`, `EvaluationCompleted`, and the rest carry real mid-operation detail instead of one flat "did it succeed" flag.

CLI SURFACE ADDED

```
field-horizon events list|show <id>|trace <correlation-id>|run <run-id>
```

B

Run manifests, registries & tiered replay

A cycle could be re-run, but nothing remembered *which* ontology, config, model digest, or embedding version had produced the original — so replay could re-execute a prompt but never honestly say what, if anything, had drifted.

7 commits · **76739c6** → **f0e9f64**

WHAT SHIPPED

- `run_manifests` — one row per meaningful run, keyed by the *same* `run_id` Phase A's `correlation_id` already used. Fingerprints (ontology, source manifest, config, dependencies, Rust binary) are content-free "did this change" hashes — cheap, and honest about only answering that one question.
- `model_registry` / `prompt_registry` / `embedding_registry` — auto-register on first sight; an unknown model is never used silently.
- Five replay tiers, each honestly scoped: **L1** re-runs the stored prompt (already existed); **L2** diffs recorded vs current environment fingerprints; **L3** reconstructs the exact evidence set from `cycle_sources`; **L4** re-runs schools clustering from its recorded seed — the one fully deterministic stage in the system; **L5** diffs a stored cycle's prompt run under its original model vs a variant, baseline against baseline.

LEVEL	QUESTION IT ANSWERS	BACKING
L1	What does this prompt produce today?	pre-existing
L2	Has the environment drifted since this ran?	<code>run_manifests</code> fingerprints
L3	What evidence actually built this prompt?	<code>cycle_sources</code>
L4	Would the same seed re-cluster the same way?	schools' own k-means seed
L5	Would a different model say this differently?	<code>model registry</code>

CLI SURFACE ADDED

```
field-horizon replay <id> --level 1..5 [--model|--variant-model]
```

```
field-horizon registry list|show model|prompt|embedding
```

C

Provenance graph & epistemic supply chain

Ancestry already existed — scattered across three ad hoc shapes: a JSON array on `cycles`, a JSON blob inside `json_entries`, and a live $O(n)$ text search for raw material. None of it was a queryable edge.

8 commits · **c1e0923** → **ef84eb5**

WHAT SHIPPED

- `provenance_edges` — one typed edge table over rows that already exist (cycle, chunk, json_entry, school, source, council), never a mirror table per object kind. A natural-key unique index makes every write idempotent for free.
- Eleven relation types actually populated — `SELECTED_BY`, `SUPPORTS` (MMR selection), `MEMBER_OF`, `SUPERSEDES` (schools), `REHABILITATES`, `RETIRES`, `PROMOTED_BY` (council), `MUTATES`, `DERIVED_FROM`, `OPPOSES`, `EXTRACTED_FROM` (ingest) — plus one deliberately *absent*: `CONTRADICTS`, because nothing in this codebase has ever written one.
- A one-shot backfill reconstructing edges from existing history, honestly counting what it couldn't attribute rather than guessing.
- The supply-chain report: a completeness score (expected links vs. actual), a synthetic-dependency ratio computed from the edges themselves — not the corpus's inconsistent `tradition` label — circular-ancestry detection, and the single weakest link in a fragment's lineage.

"Only include strategies with real backing; mark the rest as planned, not faked" — the brief's own rule. `CONTRADICTS` stayed unbuilt for the same reason it stayed unbuilt in the retrieval planner two phases later: naming an absence honestly costs nothing; faking a signal costs trust in every other number on the page.

CLI SURFACE ADDED

```
field-horizon provenance backfill|show <target>
```

```
field-horizon lineage <target>    (now renders the graph, not just genealogy)
```

D

Bitemporal canon

`cycles.verdict` is overwritten in place at every rehabilitation — so once a heresy trial changed its mind, the row itself forgot what it used to believe. Only `canon_events` JSON detail remembered.

8 commits · **7c881f4** → **b3f1f28**

WHAT SHIPPED

- `canon_temporal_states` — a purely append-only ledger of the verdict *periods* a cycle passes through. `valid_to` and `superseded_by` are deliberately *not* stored, mutated columns — they're derived at query time as "the next row for this cycle" — the one place this phase chose consistency with every other Phase A-C table over a literal reading of the brief.
- Valid-time and transaction-time travel are the *same query* over two different clocks. They agree for every live write — this engine has never backdated anything — and diverge only for a backfilled row, which is exactly the test that proves the distinction is real and not cosmetic.
- `why_changed` — resolves a cycle's transitions to the actual Phase A event and Phase C edge that caused each one, by lookup, not re-derivation.
- `school_membership_as_of` and `weather_as_of` needed *zero* schema change: `schools.run_at` and `weather_readings.recorded_at` already carried everything required.
- A sixth replay mode: re-run a cycle's prompt against canon as it stood at a past moment vs. canon as it stands now, and diff two fresh generations.

QUERY	ANSWERS
<code>canon --as-of-cycle N</code>	what counted as canon when cycle N was written
<code>canon --as-of-timestamp T</code>	what counted as canon in the modeled world at T
<code>why-changed <id></code>	which event, and which edge, changed this cycle's verdict
<code>replay --level 6 --as-of T</code>	would this prompt read differently under canon-as-of-T

CLI SURFACE ADDED

```
field-horizon canon [--as-of-cycle N | --as-of-timestamp T]
```

```
field-horizon why-changed <cycle_id>
```

```
field-horizon weather-as-of / school-membership-as-of <timestamp>
```

```
field-horizon backfill-temporal-canon
```

E

Retrieval planner v3

Hybrid retrieval v2 already existed — but as an inspection-only tool. `cycle.py` never called it; it lived behind `field-horizon retrieve` alone. The planner extends exactly that seam, and touches nothing a real cycle depends on.

6 commits · `ed4559b` → `f0ac7bd`

WHAT SHIPPED

- A rule-based classifier, not an LLM call — every trigger is a checkable fact (a declared `ontology.yaml` opposition, a term already tagged as an entity or motif, a textual match against a live canon fragment), which is what makes "picks contradiction retrieval when the query is oppositional" a test instead of a hope.
- Nine strategies considered, eight backed: `LEXICAL`, `VECTOR`, `DOMAIN_ROUTING` (v2's own baseline, always on), `ENTITY`, `MOTIF`, `CONTRADICTION`, `CANON_GENEALOGY`, `SCHOOL`, `TEMPORAL` (opt-in only, via an explicit `--as-of`).
- The full score vector the brief asked for — lexical, vector, domain prior, source weight, severity, entity/motif/contradiction signals, graph proximity, provenance quality (Phase C, reused directly), diversity contribution (the same MMR formula canon selection already used), and an `exclusion_reason` on anything a constraint removes.
- Three enforced constraints — source diversity, minimum direct evidence, maximum synthetic-evidence ratio — each excluding candidates and recording exactly why. Four more from the brief's list were named, not faked, as out of scope this phase.

The plain `retrieve` command, without `--plan`, produces byte-for-byte what it produced before this phase existed. The new planner is a second door into the same room, not a renovation of the first one.

CLI SURFACE ADDED

```
field-horizon retrieve "<query>" --plan [--as-of T] [--explain]
```



Optional seam: PrincipalContext

The one piece of the brief explicitly framed as "whenever, it's a day" — a parameter, not a subsystem.

1 commit · 9871b53

- `PrincipalContext(principal_id, roles, is_local_admin)` threads through `OperationEmitter` into a new, nullable `domain_events.principal_id` column, and through `FieldHorizonEngine.cycle()` / `.run_council()`.
- No roles, policies, permissions, or authorization logic anywhere — every operation defaults to a local administrative principal, so nothing about how the engine behaves today changed. Confirmed by running a real cycle against a copy of the production corpus: every new event carries `principal_id = "local-admin"`; every event recorded before this column existed correctly stays `NULL`, never backfilled.

HELD ACROSS ALL SIX

What stayed true from Phase A to the seam

01

Dossier before code

Every phase opened with real files and symbols cited, not inferred — and stopped for explicit authorization before implementation began.

02

Append-only, without exception

`domain_events`, `provenance_edges`, `canon_temporal_states`, `run_manifests` — none has an update or delete path. Phase D bent its own design rather than break this.

03

No silent degradation

Every write hook that can fail is try/except-and-logged, never swallowed. Missing links are counted — `unresolved_edges`, `missing_cycle_ids` — never guessed into a plausible-looking number.

04

Additive, never replacing

`search_books` / `search_json`, hybrid retrieval v2, `canon_events`, `cycles.verdict` — none were rewritten. Each phase built a layer alongside and reconciled with what already existed.

05

Real backing over stubs

`CONTRADICTS` and four of the retrieval planner's constraints were named as unbuilt, not faked. An honest gap costs nothing; a fabricated signal costs the trust in every other number on the page.

06

Live-verified, every time

Each phase's last step ran against a scratch copy of the real production corpus — not a fixture — before being called finished.

COLOPHON

Seven new modules — events, manifests, registries, provenance, temporal, retrieval_plan, principal — alongside fifteen modified. Sixty-four test files, up from forty-two. Ruff and mypy clean at every one of thirty-three commits.

6

NEW TABLES

1

NEW COLUMN

18

CLI COMMANDS ADDED